

## Лабораторная работа 4.

### Обработчики событий форм.

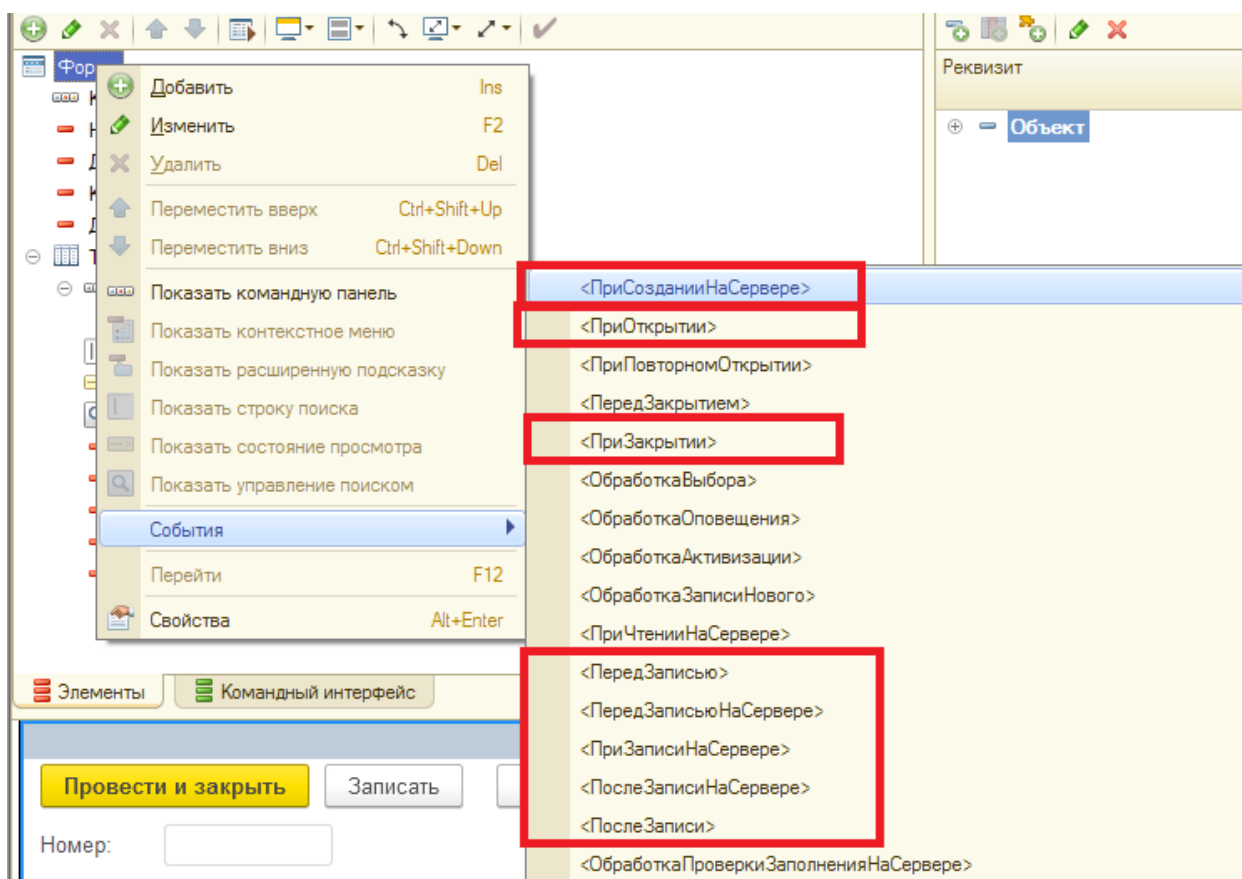
В этой лабораторной работе мы познакомимся с обработчиками событий элементов формы. Будут рассмотрены основные категории событий, в какой момент возникают события, а также как их программно обрабатывать.

**Событие** – это действие пользователей на форме, интерпретируемые системой, которые можно запрограммировать по своему усмотрению. Например, нажатие кнопки – это событие, внесение данных в поле – событие, активация строки в таблице – событие, открытие формы – событие, интерактивное сохранение данных – событие. Для каждого вида элемента формы, как и самой формы, существует значительное число событий, которыми вы почти никогда не будете пользоваться. Основные используемые уже были перечислены. О них и поговорим.

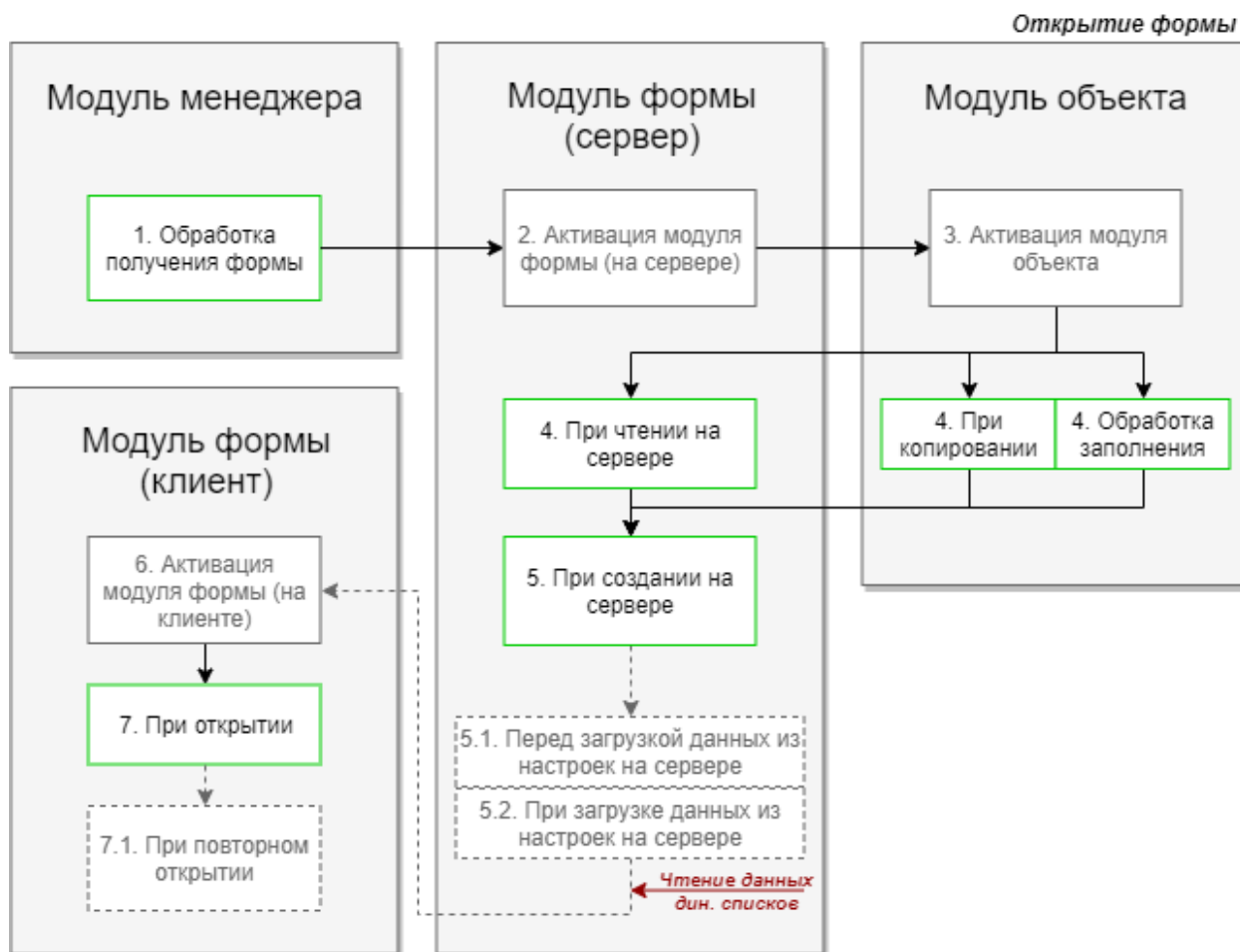
Глобально, события можно поделить на три категории: события формы, события элементов формы и кнопки. Разберем каждую из категорий.

#### СОБЫТИЯ ФОРМЫ.

Когда мы говорим о событиях формы, мы рассматриваем форму как единое целое для работы. В основном здешние события используются для задания параметров открывающейся формы, начальной настройки отображения элементов формы, а также для обработки интерактивного сохранения данных (то есть через форму, ручками). На картинке ниже **красными прямоугольниками** выделены основные обрабатываемые события.

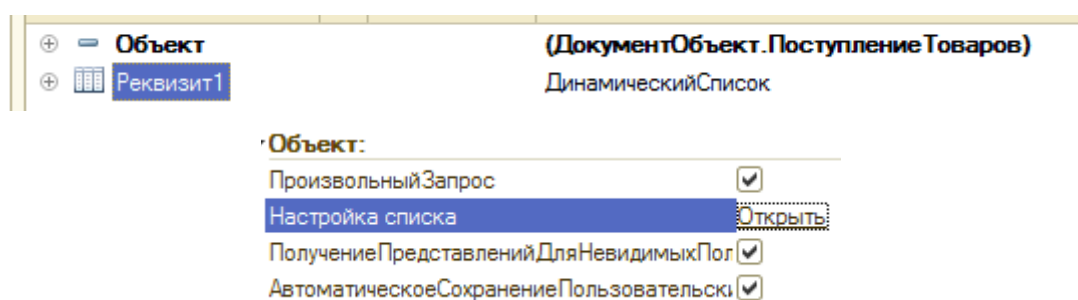


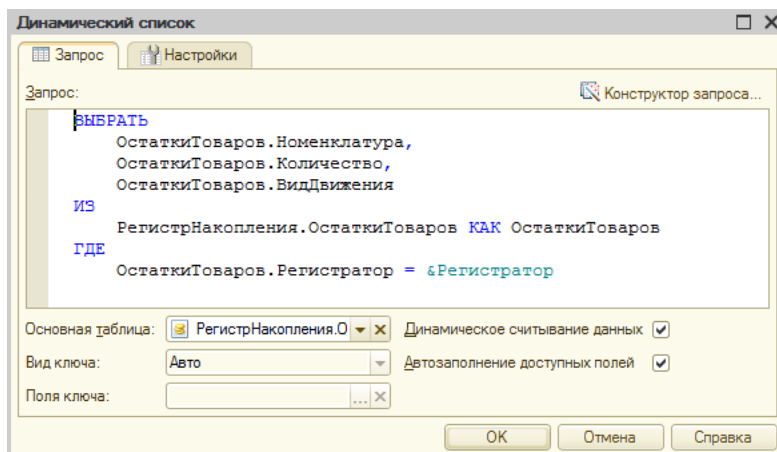
Отдельно стоит сказать по событиям *«При создании на сервере»* и *«При открытии»*. Просто надо запомнить, что любая форма создается платформой **на сервере**. Событие *«При создании на сервере»* обрабатывает именно создание формы и в основном используется для задания параметров создающейся формы. Открытие формы всегда происходит **на клиенте**, в основном используется для настройки отображения дочерних элементов формы. На картинке ниже представлен порядок действий платформы по открытию формы, запоминать не обязательно, но держать в голове надо.



Пример использования обработчика события создания формы - заполнение параметров динамического списка.

Создадим на форме реквизит с типом *«Динамический список»*, установим ему произвольный запрос, достающий информацию о поступлении товара по текущему документу поставки. Для этого установим соответствующий параметр в обработчике создания формы *«ПриСозданииНаСервере»*.





```

&НаСервере
[ Процедура ПриСозданииНаСервере (Отказ, СтандартнаяОбработка)
    Реквизит1.Параметры.УстановитьЗначениеПараметра ("Регистратор", Объект.Ссылка);
КонецПроцедуры

```

Теперь, когда вы откроете запись, в отдельном поле будут выводиться записи указанного выше запроса для конкретной записи.

| Вид движ... | Количество | Номенклатура                                   |
|-------------|------------|------------------------------------------------|
| + Приход    | 8,000      | Смартфон Samsung Galaxy A13 4 ГБ/128 ГБ черный |

Событие формы «*При открытии*». Необходимо блокировать поля для редактирования существующей записи. В данном примере, если реквизит **Номер** заполнен, то есть запись уже существует, то всем полям на форме ставится свойство **ТолькоПросмотр** в **Истину**.

Напомним, что обращение ко всем элементам формы и их свойствам доступно через **Элементы**, а чтение и запись данных формы происходит через обращение к реквизитам формы, т.е. чтобы прочитать номер, нам нужно его достать из родительского реквизита **Объект**.

```

&НаКлиенте
[ Процедура ПриОткрытии (Отказ)
    Если Объект.Номер <> "" Тогда
        Элементы.Номер.ТолькоПросмотр = Истина;
        Элементы.Дата.ТолькоПросмотр = Истина;
        Элементы.Контрагент.ТолькоПросмотр = Истина;
        Элементы.Договор.ТолькоПросмотр = Истина;
        Элементы.Товары.ТолькоПросмотр = Истина;
        Элементы.Реквизит1.ТолькоПросмотр = Истина;
    КонецЕсли;
КонецПроцедуры

```

**Поступление товаров 000000001 от 26.02.2023 19:27:04**

Номер:

Дата:

Контрагент:

Договор:

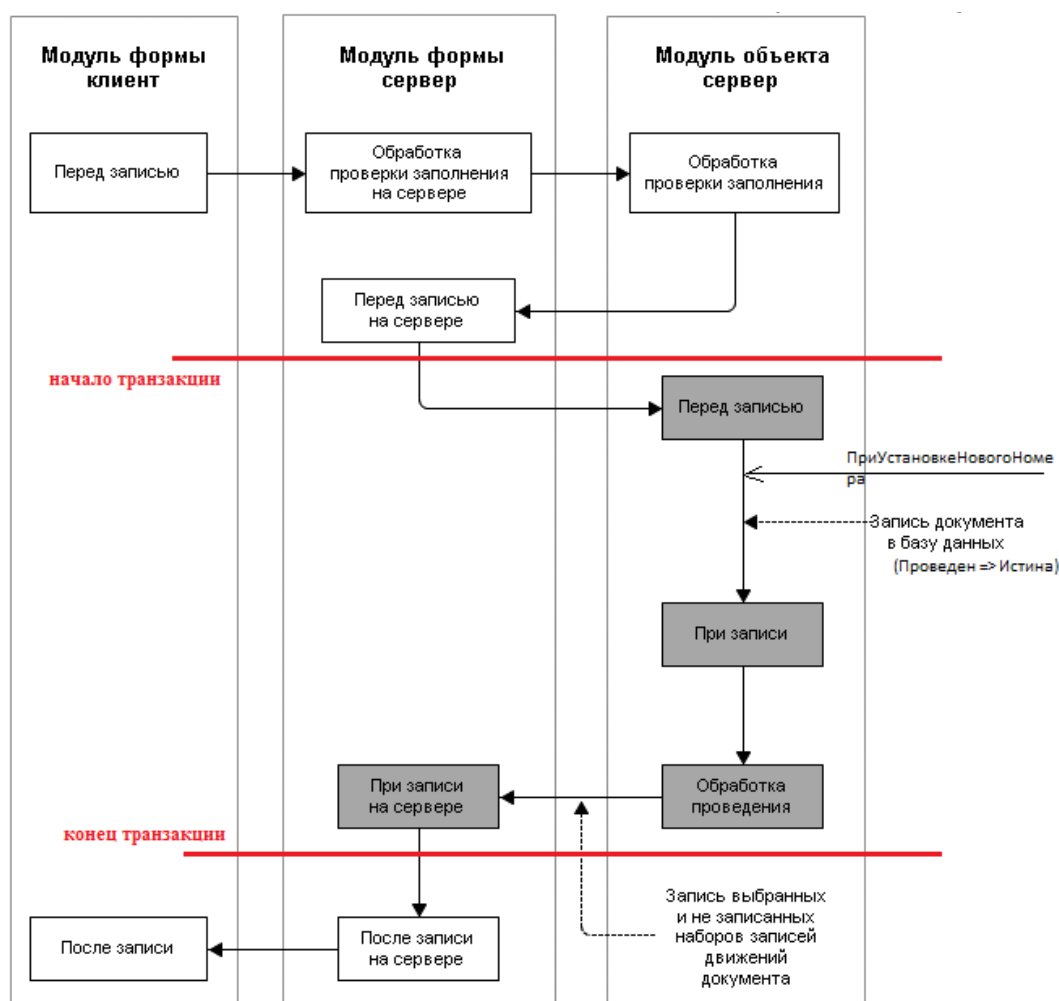
\*на картинке не видно, но поля заблокированы для изменения

События интерактивной записи. Будем проверять заполнение **наименования** через форму. Используем событие «*ПриЗаписиНаСервере*». Если в текущем изменяемом объекте пустое **наименование**, то откажем в сохранении изменений.

```

&НаСервере
□ Процедура ПриЗаписиНаСервере(Отказ, ТекущийОбъект, ПараметрыЗаписи)
    Если ТекущийОбъект.Наименование = "" Тогда
        Сообщить ("Наименование не заполнено");
        Отказ = Истина;
    КонецЕсли
КонецПроцедуры
  
```

Обработка событий записи на форме служит в основном для заполнения дополнительных свойств, для взаимодействия с реквизитами формы, для проверки заполнения полей. Ниже представлен порядок записи объекта.



Стоит почитать: <https://infostart.ru/1c/articles/1098803/> и <https://infostart.ru/1c/articles/1175475/>

## СОБЫТИЯ ЭЛЕМЕНТОВ ФОРМЫ.

Здесь мы рассмотрим события отдельных полей и таблиц. В отличие от событий формы, перечень доступных событий элементов формы намного меньше. Все события также возникают при определенных действиях пользователя на форме: изменение данных в поле, выбор из списка, активация строки таблицы и т.д. Обработчики создаются также, через ПКМ и «События». Рассмотрим наиболее используемые.

Событие *«При изменении»* срабатывает при любом изменении данных в поле. Будем делить ФИО на составные части (условимся, что части ФИО однословные). Для этого необходимо поставить обработчик на поле, которые мы будем менять, у меня ФИО отображается в элементе **Наименование**. Достаем данные из **одноименного реквизита** и делим их через пробел. Затем каждую часть разделенного ФИО записываем в свой реквизит.

```

&НаКлиенте
[ Процедура НаименованиеПриИзменении(Элемент)
    мФИО = СтрРазделить(Объект.Наименование, " ");
    ПОПЫТКА
        Объект.Фамилия = мФИО[0];
        Объект.Имя = мФИО[1];
        Объект.Отчество = мФИО[2];
    ИСКЛЮЧЕНИЕ
        КОНЕЦПОПЫТКИ;
    КонечПроцедуры

```

Событие *«Начало выбора»* срабатывает при нажатии на кнопку выбора из списка. В основном используется для передачи в список выбора собственных значений, а не всего возможного списка. Прочитав документацию по этому событию, узнаем, что для корректной работы необходимо отключить стандартную обработку. Затем с помощью дополнительной функции получим собственный список значений, из которого и будем выбирать, что записать в поле «Подразделение», на котором стоит обработчик начала выбора.

```

&НаКлиенте
[ Процедура ПодразделениеНачалоВыбора(Элемент, ДанныеВыбора, СтандартнаяОбработка)
    СтандартнаяОбработка = Ложь;
    ДанныеВыбора = сзПолучитьДанныеВыбора();
    КонечПроцедуры

[ функция сзПолучитьДанныеВыбора()
    СписокЗначений = Новый СписокЗначений;
    СписокЗначений.Добавить(Справочники.Подразделения.НайтиПоНаименованию("Подразделение 1"), "Подразделение 1");
    СписокЗначений.Добавить(Справочники.Подразделения.НайтиПоНаименованию("Подразделение 3"), "Подразделение 3");
    Возврат СписокЗначений;
    КонечФункции

```

☆ Примеров Пример Примерович (Сотрудники)

Записать и закрыть    Записать

Наименование:

Подразделение:

Дети    Образованность

Добавить

Подразделение 1

Подразделение 3

### События таблицы формы.

Вторым наиболее распространенным элементом формы является таблица, у которой также имеются события, например, изменение состава строк или активизация строки.

Создадим реквизит с типом «Таблица значений» с указанными колонками. Также создадим числовой реквизит *СуммаПоТабличке*.

ДопустимТабличка

Реквизит1

Реквизит2

Реквизит3

Реквизит4

(ТаблицаЗначений)

Строка

Булево

Число

СправочникСсылка.ФизическиеЛица

Перетащим табличку на форму (с созданием колонок). Будет примерно следующий результат.

Форма

Командная панель

СуммаПоТабличке

ДопустимТабличка

Командная панель

Строка поиска

Состояние просмотра

Управление поиском

ДопустимТабличкаРеквизит1

ДопустимТабличкаРеквизит2

ДопустимТабличкаРеквизит3

ДопустимТабличкаРеквизит4

Сумма по табличке:

Добавить

Поиск (Ctrl+F)

Поле1: Значение1

Поле2: Значение2

| Реквизит1 | Реквизит2 | Реквизит3 | Реквизит4 |
|-----------|-----------|-----------|-----------|
|           |           |           |           |

Итак, допустим нам надо пересчитать суммарную стоимость товаров в таблице. Сумма будет находиться в колонке *Реквизит3*. Для отслеживания изменения данных таблице существует событие «ПриИзменении» таблицы формы, оно будет срабатывать, когда добавились или удалились строки, а также когда изменились данные в строках.

Выбираем нашу таблицу на форме (родительский элемент), тыкаем ПКМ, «События» и выбираем событие «ПриИзменении», делаем процедуру на клиенте. Для типа данных «Таблица значений» существует метод *Итог*, получающий сумму по всей указанной колонке, то есть по *Реквизит3*. Этот итог присваиваем реквизиту *СуммаПоТабличке*.

Широко используется событие «ПриАктивизацииСтроки» таблицы формы, в основном для отображения связанных записей других таблиц формы. Для примера мы будем в отдельный реквизит писать значение Реквизита 1 выделенной строки. Для получения текущей строки воспользуемся встроенным методом *ТекущиеДанные* (допускается ситуация, когда строка не найдена, т.е. таблица пустая). Из текущей строки получаем значение колонки *Реквизит1*.

Сумма по табличке:

Добавить

| Реквизит1 | Реквизит2                | Реквизит3 | Реквизит4 |
|-----------|--------------------------|-----------|-----------|
|           | <input type="checkbox"/> | 50        |           |
|           | <input type="checkbox"/> | 80        |           |

Обратите внимание, что мы получаем значение ячейки по имени реквизита формы, а не имени элемента форму: имя реквизита формы – **Реквизит1**, имя элемента формы – **ДопустимТабличкаРеквизит1**.

```

&НаКлиенте
Процедура ДопустимТабличкаПриАктивизацииСтроки(Элемент)
    // Элемент – таблица "ДопустимТабличка" формы
    // ТекущиеДанные – текущая строка таблицы
    Если Элемент.ТекущиеДанные <> Неопределено Тогда
        ТекущееЗначениеРеквизита1 = Элемент.ТекущиеДанные.Реквизит1;
    КонецЕсли;
КонецПроцедуры

```

Текущее значение реквизита 1: ПРИМЕР

| Реквизит1 | Реквизит2                | Реквизит3 | Реквизит4 |
|-----------|--------------------------|-----------|-----------|
| ПРИМЕР    | <input type="checkbox"/> |           |           |
| fdsafdsa  | <input type="checkbox"/> |           |           |

Событие «*ПриИзменении*» также распространяется на отдельные колонки таблицы формы. Событие возникает, когда происходит изменение данных в соответствующей колонки любой из строк. Обычно используется для подсчета суммы по строке, то есть количество, умноженное на цену; для подсчета суммы НДС и прочее и прочее.

Для примера рассчитаем сумму ячеек каждой строки. Для этого сначала получим данные текущей строки через метод **ТекущиеДанные**, затем в колонку **СуммаРек3Рек5** будем писать сумму колонок **Реквизит3** и **Реквизит5**.

Обратите внимание, что один и тот же код продублирован на события изменения колонки таблицы формы **ДопустимТабличкаРеквизит3**, так и на колонку **ДопустимТабличкаРеквизит5**. Это сделано для того, чтобы сумма считалась как при изменении **Реквизита 3**, так и **Реквизита 5**.

```

&НаКлиенте
Процедура ДопустимТабличкаРеквизит3ПриИзменении(Элемент)
    СтрокаДопустимТаблички = Элементы.ДопустимТабличка.ТекущиеДанные;
    Если СтрокаДопустимТаблички <> Неопределено Тогда
        СтрокаДопустимТаблички.СуммаРек3Рек5 =
            СтрокаДопустимТаблички.Реквизит3 + СтрокаДопустимТаблички.Реквизит5;
    КонецЕсли;
КонецПроцедуры

&НаКлиенте
Процедура ДопустимТабличкаРеквизит5ПриИзменении(Элемент)
    СтрокаДопустимТаблички = Элементы.ДопустимТабличка.ТекущиеДанные;
    Если СтрокаДопустимТаблички <> Неопределено Тогда
        СтрокаДопустимТаблички.СуммаРек3Рек5 =
            СтрокаДопустимТаблички.Реквизит3 + СтрокаДопустимТаблички.Реквизит5;
    КонецЕсли;
КонецПроцедуры

```

| Р... | Ре...                    | Рекв... | Реквизит3 | Реквизит5 | Сумма 3 и 5 |
|------|--------------------------|---------|-----------|-----------|-------------|
|      | <input type="checkbox"/> |         | 80        | 60        | 140         |
|      | <input type="checkbox"/> |         | 90        | 60        | 150         |
|      | <input type="checkbox"/> |         | 34        | 66        | 100         |

Данное событие можно обрабатывать и по-другому, написав отдельную процедуру для расчета, к которой будут обращаться процедуры обработчика.

```

«НаКлиенте
□ Процедура ДопустимТабличкаРеквизит3ПриИзменении ( Элемент)
    РассчитатьСумму35 ();
    КонечПроцедуры

«НаКлиенте
□ Процедура ДопустимТабличкаРеквизит5ПриИзменении ( Элемент)
    РассчитатьСумму35 ();
    КонечПроцедуры

«НаКлиенте
□ Процедура РассчитатьСумму35 ()
    СтрокаДопустимТаблички = Элементы.ДопустимТабличка.ТекущиеДанные;
    Если СтрокаДопустимТаблички <> Неопределено Тогда
        СтрокаДопустимТаблички.СуммаРек3Рек5 =
            СтрокаДопустимТаблички.Реквизит3 + СтрокаДопустимТаблички.Реквизит5;
    КонечЕсли;
    КонечПроцедуры

```

Можно пойти еще дальше, назначив один и тот же обработчик разным полям. Это позволяет, но редко когда используется. Для этого нужно как обычно создать обработчик для одного из полей, и назначить этот же обработчик в событие другого поля.

Обработчик изменения поля *Реквизит 3* -

|              |                                       |
|--------------|---------------------------------------|
| ▼ События:   |                                       |
| ПриИзменении | ДопустимТабличкаРеквизит3ПриИзменении |

Обработчик изменения поля *Реквизит 5* -

|              |                                       |
|--------------|---------------------------------------|
| ▼ События:   |                                       |
| ПриИзменении | ДопустимТабличкаРеквизит3ПриИзменении |

Процедура обработчика -

```

«НаКлиенте
□ Процедура ДопустимТабличкаРеквизит3ПриИзменении ()
    СтрокаДопустимТаблички = Элементы.ДопустимТабличка.ТекущиеДанные;
    Если СтрокаДопустимТаблички <> Неопределено Тогда
        СтрокаДопустимТаблички.СуммаРек3Рек5 =
            СтрокаДопустимТаблички.Реквизит3 + СтрокаДопустимТаблички.Реквизит5;
    КонечЕсли;
    КонечПроцедуры

```

## КНОПКИ.

Что же по кнопкам, то здесь все максимально просто, у них одно событие – нажатие. Нет каких-то стандартных действий для кнопок, они используются для всего: для заполнения полей, таблиц, для создания новых записей базы данных, для создания табличных документов, для обращений к WEBу. В общем, вы ничем морально не ограничены в обработке нажатий кнопок.

Этот пример ничего важного не делает, по кнопке перезаполняет таблицу случайными значениями.

```

&НаКлиенте
[ Процедура Команда1(Команда)
    Команда1НаСервере();
КонецПроцедуры

&НаСервере
[ Процедура Команда1НаСервере()
    ДопустимТабличка.Очистить();
    ГСЧ = Новый ГенераторСлучайныхЧисел;
    Для а = 1 по 5 Цикл
        НоваяСтрока = ДопустимТабличка.Добавить();
        НоваяСтрока.Реквизит1 = Строка(Новый УникальныйИдентификатор());
        НоваяСтрока.Реквизит2 = ГСЧ.СлучайноеЧисло(0, 1) = 0;
        НоваяСтрока.Реквизит3 = ГСЧ.СлучайноеЧисло(1, 9);
    КонецЦикла;
КонецПроцедуры

```

| Реквизит1                            | Реквизит2                           | Реквизит3 |
|--------------------------------------|-------------------------------------|-----------|
| 277c7ba8-c6f7-41d7-877c-e2af1da2bb89 | <input checked="" type="checkbox"/> | 8         |
| cfd020db-a40c-4357-b05a-13bc49779f51 | <input type="checkbox"/>            | 9         |
| 29979876-f4d5-409f-851a-611154397a0  | <input type="checkbox"/>            | 1         |
| a0bd14b5-b614-4ded-89e5-8b480c0108de | <input type="checkbox"/>            | 4         |
| 478f262d-60f3-45ff-a31d-3de0df09ede0 | <input checked="" type="checkbox"/> | 9         |

## ЗАДАНИЕ.

Для своего варианта создать по **два** обработчика событий каждого вида, представленного в лабораторной работе, а именно:

- События формы:
  - Событие создания формы
  - Событие открытия формы
  - События интерактивной записи (любое событие)
- События элементов формы:
  - Событие изменения поля
  - Событие начала выбора
  - Событие таблицы формы – при изменении состава строк
  - Событие таблицы формы – при активизации строки
  - Событие таблицы формы – при изменении значения колонки
- События кнопок – нажатие.

**ПРИМЕЧАНИЕ.**

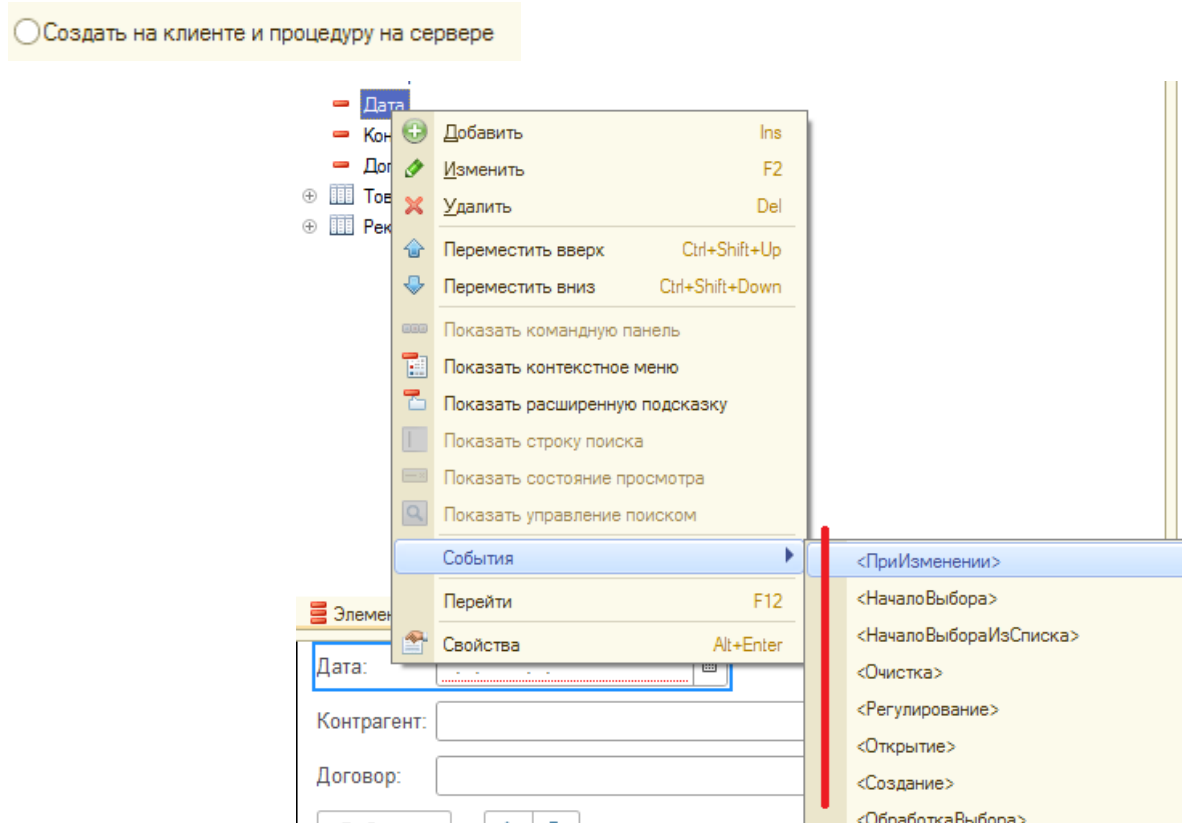
КАК РЕКОМЕНДУЕТСЯ И НЕ РЕКОМЕНДУЕТСЯ РАБОТАТЬ С ОБРАБОТЧИКАМИ СОБЫТИЙ.

С точки зрения программирования обработчиков событий вы ограничены только требованиями Платформы 1С. Ну то есть нельзя делать запросы на клиенте, и показывать пользователю excel на сервере, и т.п. Однако это не означает, что надо всю логику делать в одном обработчике.

Помните, что обработчик события – это обработчик ОДНОГО события. Если нужно, чтобы значение одного поля зависело от значения другого, чтобы в зависимости от роли пользователя были видны различные поля, чтобы проверить заполнение полей – это пожалуйста. Как бы вы не хотели, но по одной кнопке «Сделать все хорошо» нельзя сделать все хорошо, для этого существует комплекс кнопок.

**КАК НАЧАТЬ ПРОГЛАТЬ ОБРАБОТЧИК СОБЫТИЯ? -**

Выбрать нужный элемент на форме, нажать ПКМ, выбрать «События», выбрать событие. В зависимости от события надо выбрать, где будет выполняться код. В подавляющем большинстве -



**ГДЕ ПИСАТЬ КОД?** - В модуле формы.

**КАК ПЕРЕЙТИ В МОДУЛЬ ФОРМЫ?** - Снизу на форме вкладка «Модуль».

**А ЧТО ПИСАТЬ?** - То, что вам нужно по заданию. См. теорию.

**Я НИЧЕГО НЕ ПОНИМАЮ!** - Спроси преподавателя, или загугли, на крайняк – спроси соседа.

**СОСЕД ТОЖЕ НИЧЕГО НЕ ПОНИМАЕТ! / НЕ ГУГЛИТСЯ!** - Спроси преподавателя.

**А ЧТО ВООБЩЕ ПРОИСХОДИТ?** - Основы разработки 1С